오즈코딩스쿨 × 고용노동부 · 1인 창업가 개발부트캠프 7기

Git & GitHub

DAY 07

Git & GitHub 개념과 협업 실습

코드의 변경 이력을 기록하고 되돌리는 방법을 익힙니다.
수업 폴더를 GitHub에 올리고, 포트폴리오는 GitHub Pages로 공개합니다.

강사 김태민

TODAY'S GOAL

오늘 끝나면 할 수 있는 것

1. **Git으로 코드 변경 이력을 관리할 수 있다**

   add와 commit으로 작업을 기록하고 되돌립니다.

2. **GitHub에 올리고 주소로 공개할 수 있다**

   oz-lecture 저장소에 push하고, 포트폴리오는 GitHub Pages로 배포합니다.

3. **상황에 맞는 Git 사용 전략을 고를 수 있다**

   브랜치 · merge · conflict · PR이 어떤 약속 안에서 쓰이는지 이해합니다.

오늘의 결과물

GitHub 저장소 두 개(oz-lecture · portfolio)와 포트폴리오 공개 주소 하나

ROADMAP

오늘의 흐름

01

Git이란

변경 이력
세 공간

02

기본 명령어

add · commit
실습 01

실습 포함

03

GitHub와 Pages

push와 공개
실습 02 · 03

실습 포함

04

Git 사용 전략

브랜치 · 워크플로
merge · PR

앞의 두 섹션은 내 컴퓨터 안에서만 일어나는 일입니다.
세 번째 섹션부터 인터넷에 올라가고, 네 번째에서 함께 쓰는 약속을 봅니다.

되짚기 · 01

1챕터에서 만든 것

1	**DAY 01~03 · 구조**	시맨틱 태그로 문서의 뼈대를 만들었습니다
2	**DAY 04~06 · 스타일**	선택자, 박스 모델, Flexbox와 Grid, 반응형
3	**DAY 06 · 마무리**	점검하고 AI 리뷰를 반영해 1챕터 최종본(about-me.html)을 만들었습니다
4	**지금 상태**	oz-lecture 폴더에 날마다 실습이 쌓였고, 모두 내 컴퓨터 안에만 있습니다

오늘은 oz-lecture를 GitHub에 올리고, 최종본은 포트폴리오로 따로 공개합니다

되짚기 · 02

지금까지는 이렇게 관리했습니다

```
about-me.html
about-me_수정.html
about-me_수정2.html
about-me_최종.html
about-me_최종_진짜최종.html

/* 어느 것이 최신인지, 무엇이 달라졌는지
   일주일 뒤에는 알 수 없습니다 */
```

무엇이 문제인가

파일이 늘어나기만 하고
언제 무엇을 왜 바꿨는지는
어디에도 남지 않습니다.

되돌릴 수도 없습니다

어제 잘 되던 코드로 돌아가려면
기억에 의존해야 합니다.

준비 · 다 같이

Git을 처음 한 번 셋팅합니다

```
$ git --version
git version 2.x.x        ← 나오면 설치돼 있습니다

$ git config --global user.name "김코딩"
$ git config --global user.email "me@example.com"
$ git config --global init.defaultBranch main
$ git config --global pull.rebase false   ← pull 할 때 합치는 방식
$ git config --list        ← 확인
```

준비할 것

Git 설치
GitHub 계정 (이메일 인증까지)
oz-lecture 폴더
터미널은 VS Code에서
Windows는 Ctrl + `
Mac은 Command + J

user.email은 GitHub 계정의 메일과 같게 적습니다

커밋에 남는 이름과 메일이 이 값입니다. 같은 메일이면 GitHub에서 기록이 내 계정과 연결됩니다(23장).

01

Git이란 무엇인가

변경 이력 · Git과 GitHub · 세 개의 공간

Git은 변경 이력을 기록하는 도구입니다

파일 이름으로 관리

복사본이 쌓입니다

바꿀 때마다 파일을 복사합니다.
무엇이 달라졌는지는 남지 않습니다.
되돌리려면 기억에 의존합니다.

Git으로 관리

한 파일에 이력이 쌓입니다

파일은 하나만 둡니다.
바뀐 내용과 이유가 기록으로 남습니다.
어느 시점으로든 되돌릴 수 있습니다.

사진첩이 아니라 일기장에 가깝습니다

결과물만 모으는 것이 아니라 언제 무엇을 왜 바꿨는지를 함께 남깁니다.

파일 이름으로 관리하면 이렇게 됩니다



어느 것이 최신인가요

날짜와 이름을 붙여 두어도
며칠 지나면 알 수 없습니다.

무엇이 왜 달라졌나요

두 파일을 나란히 열어
눈으로 비교하는 수밖에 없고,
바꾼 이유는 어디에도
남지 않습니다.

버전 관리가 해주는 일은 셋입니다

01

협업

여러 사람이 같은 코드를
동시에 고칠 수 있습니다.
누가 무엇을 고쳤는지
섞이지 않습니다.

02

복구

실수해도 이전 버전으로
돌아갈 수 있습니다.
어제 잘 되던 상태가
그대로 남아 있습니다.

03

추적

누가 언제 무엇을 왜
바꿨는지 확인할 수
있습니다.

혼자 만들 때도 복구와 추적은 그대로 필요합니다

"어제 수정이 최신인가요?" "이 파일의 원본이 뭐죠?" 라는 질문이 사라집니다.

01 · 구분

Git과 GitHub는 다른 것입니다

Git

내 컴퓨터의 프로그램

변경 이력을 기록하는 도구입니다.
인터넷이 없어도 동작합니다.
만든 사람은 리누스 토르발스,
2005년에 공개했습니다.

GitHub

그 기록을 올려두는 웹 서비스

Git 저장소를 인터넷에 보관합니다.
다른 사람과 함께 보고 고칩니다.
비슷한 서비스로 GitLab,
Bitbucket도 있습니다.

Git을 배우면 GitHub가 아닌 곳에서도 그대로 쓸 수 있습니다

01 · 특징

Git은 이력을 내 컴퓨터에 전부 둡니다

예전 방식 — SVN

서버에만 이력이 있습니다

서버가 고장 나면 작업이 멈춥니다.
인터넷이 끊겨도 기록을 남길 수 없습니다.

Git — 분산형

내 컴퓨터에도 전부 있습니다

인터넷 없이도 커밋할 수 있습니다.
빠르고, 서버 사고에도 이력이 남습니다.

그래서 오늘 앞의 두 섹션은 인터넷 없이도 전부 실습할 수 있습니다

GitHub에 올리는 것은 세 번째 섹션부터입니다.

파일은 세 개의 공간을 지나갑니다

01 작업 폴더

Working Directory

지금 파일을 고치는 자리입니다.
여기서 저장하면 파일만 바뀌고
기록은 남지 않습니다.

02 스테이지

Staging Area

이번 기록에 담을 것만 골라두는
대기 자리입니다.
add로 여기에 올립니다.

03 저장소

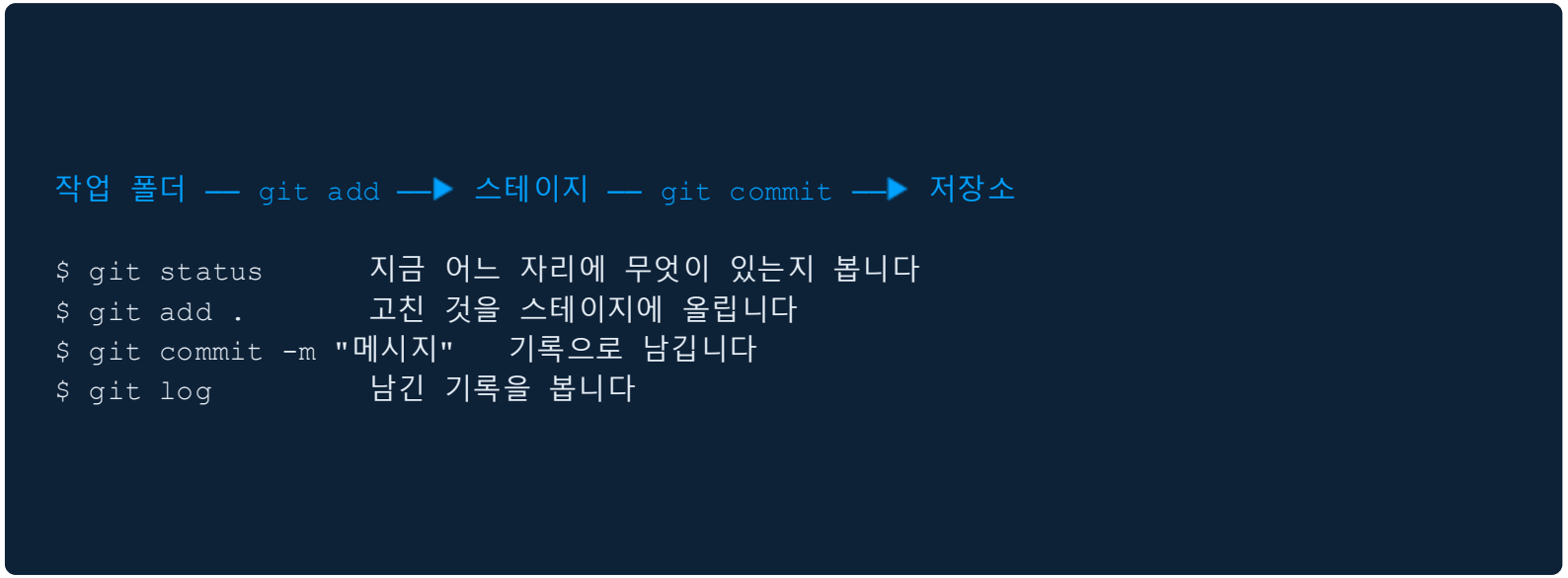
Repository

기록이 실제로 쌓이는 자리입니다.
commit으로 여기에 남깁니다.
한 번 남기면 되돌릴 수 있습니다.

고친 것 전부가 아니라 고를 수 있다는 점이 핵심입니다

세 파일을 고쳤어도 그중 하나만 골라 기록으로 남길 수 있습니다. 스테이지가 그 역할을 합니다.

세 공간을 지나가는 명령어



여기서 쓰는 것은 넷

status, add,
commit, log 넷입니다.
뒤에서 push · pull과
브랜치가 더해집니다.

명령어를 외우려 하지 말고 지금 파일이 어느 자리에 있는지 생각합니다

status는 그 자리를 알려주는 명령어라서 가장 많이 칩니다.

여기까지 정리

1	**Git**	내 컴퓨터에서 변경 이력을 기록하는 프로그램입니다
2	**GitHub**	그 기록을 인터넷에 올려두고 함께 보는 서비스입니다
3	**세 공간**	작업 폴더 → 스테이지 → 저장소 순서로 지나갑니다
4	**핵심**	고친 것 중에서 무엇을 기록으로 남길지 고를 수 있습니다

다음은 실제로 명령어를 쳐봅니다

02

기본 명령어

init · status · add · commit · 실습 01

init — 이 폴더를 저장소로 만듭니다

```
$ cd oz-lecture
$ git init
Initialized empty Git repository in …/oz-lecture/.git/

$ ls -a
.   ..   .git   .gitignore   Day01   Day02   …   Day06

/* .git 폴더가 이력을 담는 자리입니다 */
```

한 폴더에 한 번만

.git이 이미 있으면
다시 하지 않습니다.
폴더마다 따로 만듭니다.

.git을 지우면

이력이 전부 사라집니다.
그 폴더는 그냥 폴더가 됩니다.

status — 지금 상태를 보여줍니다

```
$ git status
On branch main
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Day01/   Day02/   …   Day06/
nothing added to commit but untracked files present
```

가장 많이 치는 명령어

막히면 일단 status입니다.
지금 어느 자리에 무엇이
있는지 영어로 알려줍니다.

Untracked 는 아직 Git이 추적하지 않는 파일입니다

add 하는 순간부터 추적이 시작되고, 그때부터 변경 내용이 기록됩니다.

add — 기록에 담을 것을 고릅니다

```
$ git add Day06/about-me.html      파일 하나만
$ git add .                        고친 것 전부

$ git status
Changes to be committed:
        new file:   Day01/first.html
/* 초록색으로 바뀌면 스테이지에 올라간 것입니다 */
```

add는 저장이 아닙니다

아직 기록으로 남은 것이
아니라 담아둔 상태입니다.
여기까지는 언제든
되돌릴 수 있습니다.

잘못 담았다면 git restore --staged 파일명 으로 다시 내립니다

add 하기 전에 git status 로 무엇이 담기는지 확인하는 습관을 함께 붙입니다.

commit — 기록으로 남깁니다

```
$ git commit -m "DAY01~06 수업 파일 추가"
[main (root-commit) a1b2c3d] DAY01~06 수업 파일 추가
 21 files changed, 847 insertions(+)

$ git log --oneline
a1b2c3d DAY01~06 수업 파일 추가
/* a1b2c3d 가 이 기록의 이름입니다 */
```

-m 은 메시지입니다

무엇을 왜 바꿨는지 적습니다.
생략하면 편집기가 열립니다.
앞의 일곱 자리는
긴 이름을 줄여 쓴 것입니다.

커밋을 남긴 순간부터 그 상태가 보존됩니다

파일을 아무리 고쳐도 이 시점으로 돌아올 수 있습니다. 이것이 Git을 쓰는 이유입니다.

02 · 확인

터미널에서는 이렇게 보입니다

`git init → git add . → git commit -m "init project"`

```
DEVELOPER-01@DESKTOP-LO8QNMQ MINGW64 /e/98.developer-private/3.lecture/projects/oz-lecture
$ git init
Initialized empty Git repository in E:/98.developer-private/3.lecture/projects/oz-lecture/.git/

DEVELOPER-01@DESKTOP-LO8QNMQ MINGW64 /e/98.developer-private/3.lecture/projects/oz-lecture (main)
$ git add .

DEVELOPER-01@DESKTOP-LO8QNMQ MINGW64 /e/98.developer-private/3.lecture/projects/oz-lecture (main)
$ git commit -m "init project"
[main (root-commit) b35ecd4] init project
 14 files changed, 526 insertions(+)
 create mode 100644 day01/first.html
 create mode 100644 day01/second.html
 create mode 100644 day02/form.html
 create mode 100644 day02/multimedia.html
 create mode 100644 day02/semantic.html
 create mode 100644 day03/my-website.html
 create mode 100644 day03/styles.css
 create mode 100644 day04/my-view.html
 create mode 100644 day04/styles.css
 create mode 100644 day05/meida/index.html
 create mode 100644 day05/meida/styles-desktop-first.css
 create mode 100644 day05/meida/styles-mobile-first.css
 create mode 100644 day05/meida/styles.css
 create mode 100644 day07/bootstrap/index.html
```

세 명령을 차례로 실행한 결과 — 1기 수업 화면

commit 뒤에 나오는 create mode 목록이 이번에 담긴 파일입니다

커밋 메시지는 미래의 나에게 쓰는 것입니다

이렇게 쓰면

나중에 못 찾습니다

```
수정
ㅇㅇ
최종
```

무엇이 바뀌었는지 알 수 없습니다.

이렇게 쓰세요

무엇을 왜 바꿨는지

```
소개 섹션에 프로필 사진 추가
카드 겹치는 문제 수정
안 쓰는 CSS 규칙 정리
```

한 줄로 무엇을 했는지 적습니다.

한 커밋에는 한 가지 일만 담습니다

log — 남긴 기록을 봅니다

```
$ git log
commit a1b2c3d… (HEAD -> main)
Author: 김코딩 <me@example.com>
Date:   Tue Oct 20 10:12:00 2026 +0900
    DAY01~06 수업 파일 추가

$ git log --oneline      한 줄씩 짧게
```

Author에 남는 값

오늘 처음에 설정한 user.name과 user.email입니다. GitHub 계정과 같은 메일이면 기록이 계정과 연결됩니다.

화면이 멈춘 것처럼 보이면 q 를 누릅니다

기록이 길어지면 한 화면씩 보여주는 상태가 됩니다. 오류가 아닙니다.

02 · .gitignore

올리면 안 되는 것은 미리 걸러냅니다

```
# .gitignore
.DS_Store           # 맥이 만드는 숨김 파일
node_modules/       # 설치한 라이브러리
.env                # 비밀번호와 열쇠
*.log               # 로그 파일

/* 아직 추적 전인 파일은 여기 적으면 add에서 빠집니다 */
```

특히 중요한 것

비밀번호, API 키 같은 값은 한 번 올라가면 기록에 남습니다. 파일에서 지워도 이력에서 되살릴 수 있습니다.

.gitignore를 먼저 만들고 그다음 add 합니다

이미 추적 중인 파일은 .gitignore에 적어도 계속 따라옵니다. git rm --cached 파일명 을 한 번 실행해야 빠집니다.

실습 01

oz-lecture를 Git 저장소로 만들기

DAY 01부터 실습한 oz-lecture 폴더를 저장소로 만들어 첫 기록을 남기고, 오늘 Day07 폴더를 더해 한 번 더 기록합니다.

```
$ cd oz-lecture
$ git init
# .gitignore 파일을 만들고 .DS_Store 한 줄
$ git status
$ git add .
$ git commit -m "DAY01~06 수업 파일 추가"
# Day07 폴더와 오늘 파일을 만든 뒤
$ git add .
$ git commit -m "DAY07 폴더 추가"
$ git log --oneline
```

확인할 것

.git이 oz-lecture 바로 안에 있다

log에 커밋이 두 개 있다

메시지가 한 줄로 읽힌다

막히면

git status 를 먼저 칩니다.

실습 01 · 점검

여기서 자주 막힙니다

01

폴더가 다릅니다

터미널의 현재 위치가 oz-lecture가 맞는지 봅니다. pwd로 확인하고 cd로 옮깁니다.

02

이름을 안 정했습니다

Please tell me who you are 가 나오면 오늘 처음의 config 명령을 다시 실행합니다.

03

편집기가 열렸습니다

-m 을 빠뜨리면 편집기가 열립니다. 메시지를 적고 저장한 뒤 닫으면 커밋이 완료됩니다.

log에서 멈춘 것 같으면 q, 편집기가 vim이면 :wq 로 빠져나옵니다

03

GitHub 원격 저장소

push · pull · README · GitHub Pages · 실습 02 · 03

03 · 개념

원격 저장소는 인터넷에 둔 복사본입니다

로컬 저장소

내 컴퓨터의 .git

지금까지 만든 기록이 있는 자리입니다.
내 컴퓨터가 고장 나면 함께 사라집니다.

원격 저장소

GitHub의 저장소

같은 기록을 인터넷에 둡니다.
다른 컴퓨터에서도 받을 수 있고,
다른 사람도 볼 수 있습니다.

push는 올리기, pull은 내려받기입니다

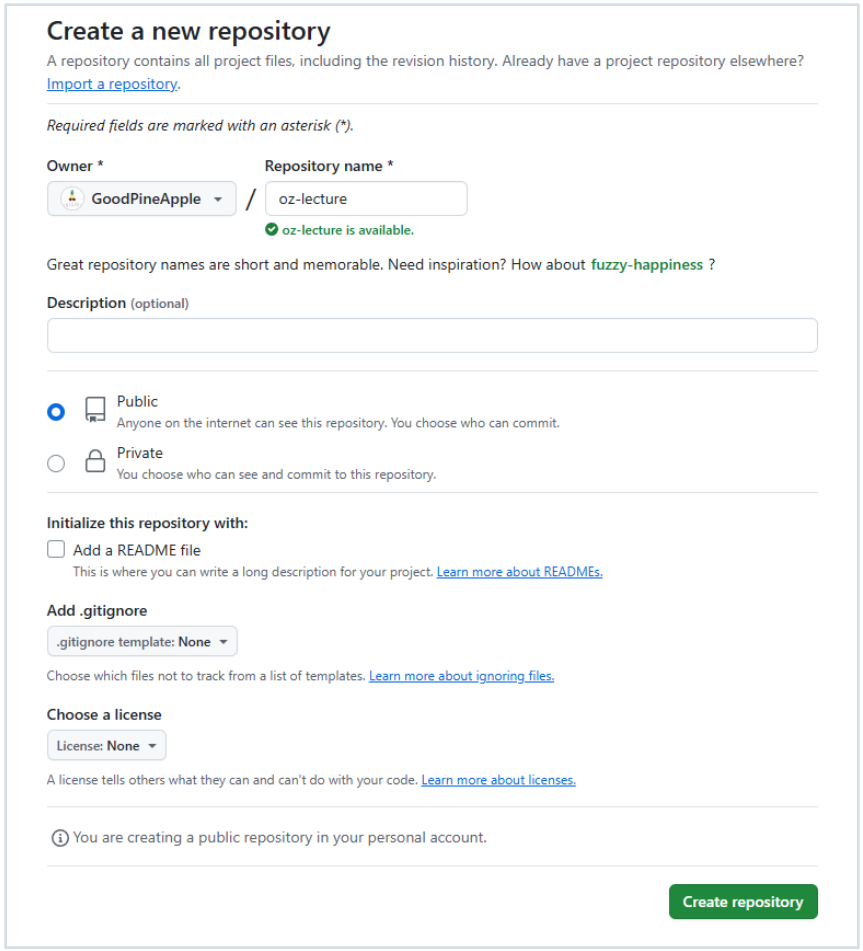
두 저장소는 자동으로 같아지지 않습니다. 내가 직접 올리고 내려받아야 합니다.

GitHub에서 저장소를 만듭니다

1. **01** 오른쪽 위 + 버튼 → New repository
2. **02** 이름은 영어 소문자와 하이픈으로 (예: oz-lecture)
3. **03** Public을 고릅니다. 강사와 동료가 링크로 볼 수 있습니다
4. **04** README와 .gitignore는 지금은 체크하지 않습니다
5. **05** Create repository를 누르면 안내 화면이 나옵니다

이미 로컬에 저장소가 있으므로 안내 화면의 두 번째 상자를 따라 칩니다

저장소 생성 화면입니다



확인할 곳 셋

Repository name
Public 선택
Add a README file 체크 해제

화면은 바뀔 수 있습니다

GitHub 화면은 종종 달라집니다.
항목 이름을 기준으로
찾으면 됩니다.

03 · push

원격 저장소와 연결하고 올립니다

```
$ git remote add origin https://github.com/사용자명/oz-lecture.git
$ git branch -M main
$ git push -u origin main

/* origin  = 원격 저장소에 붙인 별명 */
/* -u      = 앞으로는 git push 만 쳐도 되게 기억 */
$ git remote -v      연결 확인
```

처음 한 번만

remote add는 저장소마다 한 번 합니다.
-u 는 새 브랜치를 처음 올릴 때마다 붙입니다.

주소를 잘못 적었다면 git remote set-url origin 새주소 로 고칩니다

push할 때 로그인을 물어봅니다

01

비밀번호는 안 됩니다

GitHub는 2021년부터
계정 비밀번호로 하는
인증을 받지 않습니다.

02

토큰을 씁니다

우측 상단 프로필 → Settings
→ 맨 아래 Developer Settings
→ Personal access tokens
→ repo 체크 후 생성

03

더 편한 방법

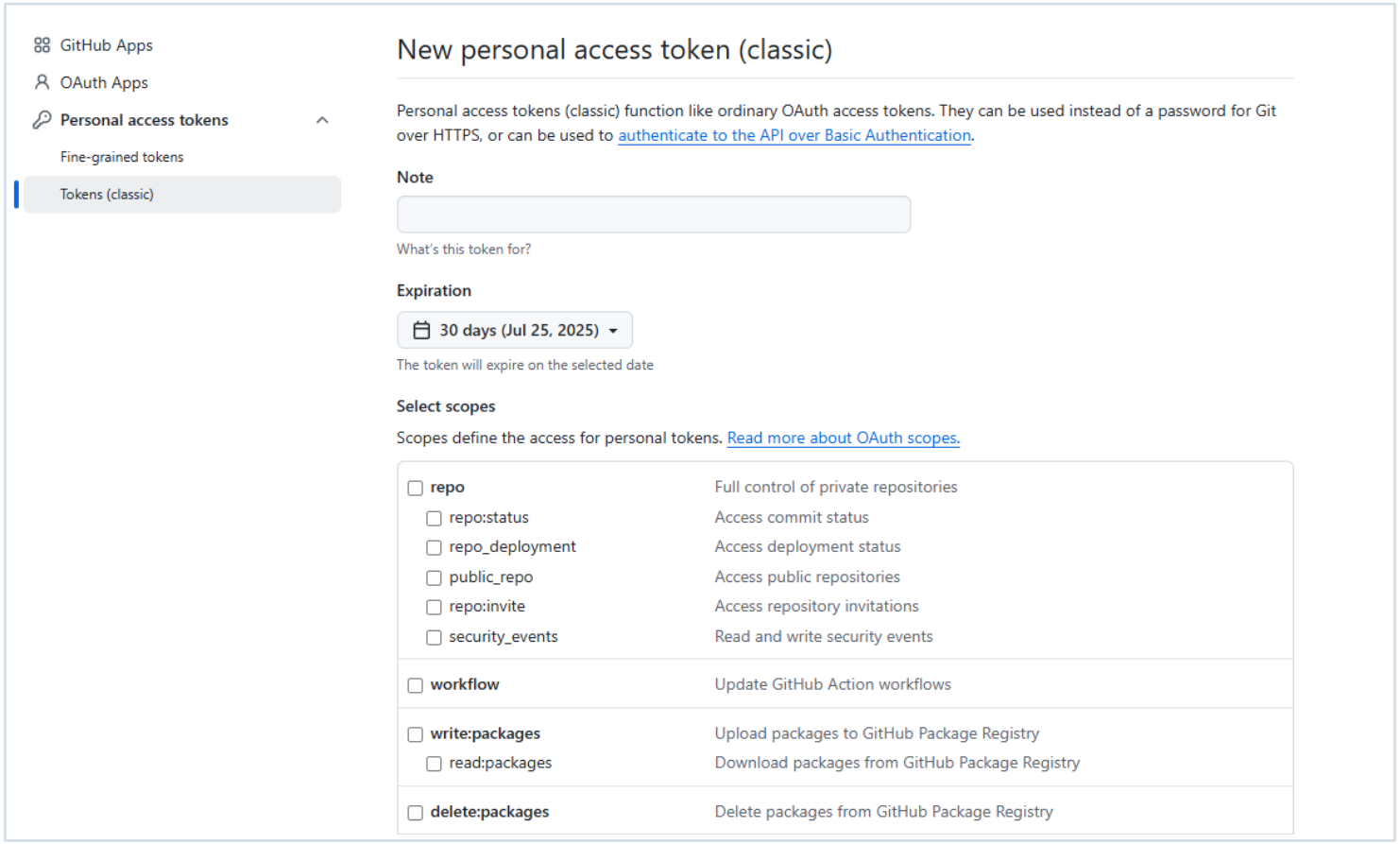
GitHub CLI를 설치하고
gh auth login 을 하면
브라우저로 한 번에
끝납니다.

발급한 토큰은 화면을 닫으면 다시 볼 수 없습니다

안전한 곳에 복사해 두게 하고, 절대 코드나 저장소에 적지 않도록 안내합니다.

토큰은 이 화면에서 만듭니다

Settings → Developer settings → Personal access tokens → Tokens (classic)



체크할 것

scopes 목록에서 repo 하나만 체크하면 이번 실습에는 충분합니다.

한 번만 보입니다

만든 직후 화면을 벗어나면 토큰을 다시 볼 수 없습니다. 복사해서 따로 보관하게 하세요.

받아오는 명령어는 두 가지입니다

```
$ git fetch       원격의 최신 이력만 받아옵니다 (내 파일은 그대로)
$ git pull        가져와서 내 파일에 바로 합칩니다
/* pull = fetch + merge */

$ git clone https://github.com/사용자명/oz-lecture.git
/* 남의 저장소나 내 저장소를 통째로 처음 받아올 때 */
```

언제 쓰나요

다른 컴퓨터에서 작업했거나
함께 하는 사람이 올렸을 때
먼저 pull하고 시작합니다.

작업을 시작할 때 pull, 끝낼 때 push 를 습관으로 만듭니다

두 사람이 같은 저장소를 쓰면 이렇게 됩니다

```
A                    원격 저장소                    B
──────────────────────────────────────────────────
1. push        →     first commit
                     second commit
2.                   같은 이력          →     clone
3.                   new commit         ←     push
4. pull        ←     new commit
/* A가 pull 하지 않고 push 하면 거절당합니다 */
```

순서를 습관으로

작업을 시작할 때 pull,
끝낼 때 push.
이 순서만 지키면
충돌이 크게 줄어듭니다.

혼자서도 컴퓨터가 두 대라면 똑같은 상황이 생깁니다

README.md는 저장소의 첫 화면입니다

```
# oz-lecture
1인 창업가 개발부트캠프 7기 수업 실습 폴더입니다.

## 들어 있는 것
- Day01~03  HTML 문서 구조 · 폼 · 시맨틱
- Day04~06  CSS 선택자 · 레이아웃 · 반응형
- Day07     Git과 GitHub

## 포트폴리오
https://사용자명.github.io/portfolio/
```

마크다운입니다

.md로 끝나는 파일은
마크다운 형식입니다.
#은 제목, -는 목록입니다.

무엇을 적나요

무엇을 만들었는지,
무엇으로 만들었는지,
어디서 볼 수 있는지.

실습 02

oz-lecture를 GitHub에 올리기

GitHub에 oz-lecture 저장소를 만들어 연결하고, README를 쓴 뒤 다시 push합니다.
저장소 주소를 채팅창에 공유합니다.

해야 할 것

GitHub에 oz-lecture 저장소 만들기

remote add로 연결하기

push해서 파일 올리기

README.md 쓰고 다시 push

확인할 것

저장소 화면에 Day01~Day07 폴더가 보인다

README가 첫 화면에 나온다

커밋 목록에 세 개가 있다

앞으로는 수업이 끝날 때마다 그날 폴더를 커밋하고 push합니다

포트폴리오는 저장소를 따로 만듭니다

```
바탕화면/   (문서 폴더여도 됩니다)
├── oz-lecture/          ← 저장소 1 : 수업 실습
│   ├── Day01/ … Day07/
│   └── README.md
└── portfolio/           ← 저장소 2 : 공개할 포트폴리오
    ├── index.html       ← Day06 about-me.html의 새 이름
    ├── style.css
    └── me.jpg
```

왜 따로 두나요

Pages 주소는 저장소마다
하나씩 생깁니다.
보여 줄 것만 담은 저장소가
포트폴리오로 쓰기 좋습니다.

oz-lecture 안에 만들지 않고, 첫 화면 파일 이름은 index.html로 바꿉니다

저장소 안에 저장소를 두면 바깥 저장소에 안쪽 파일이 담기지 않습니다. Pages는 index.html을 첫 화면으로 찾습니다.

GitHub Pages로 주소를 만듭니다

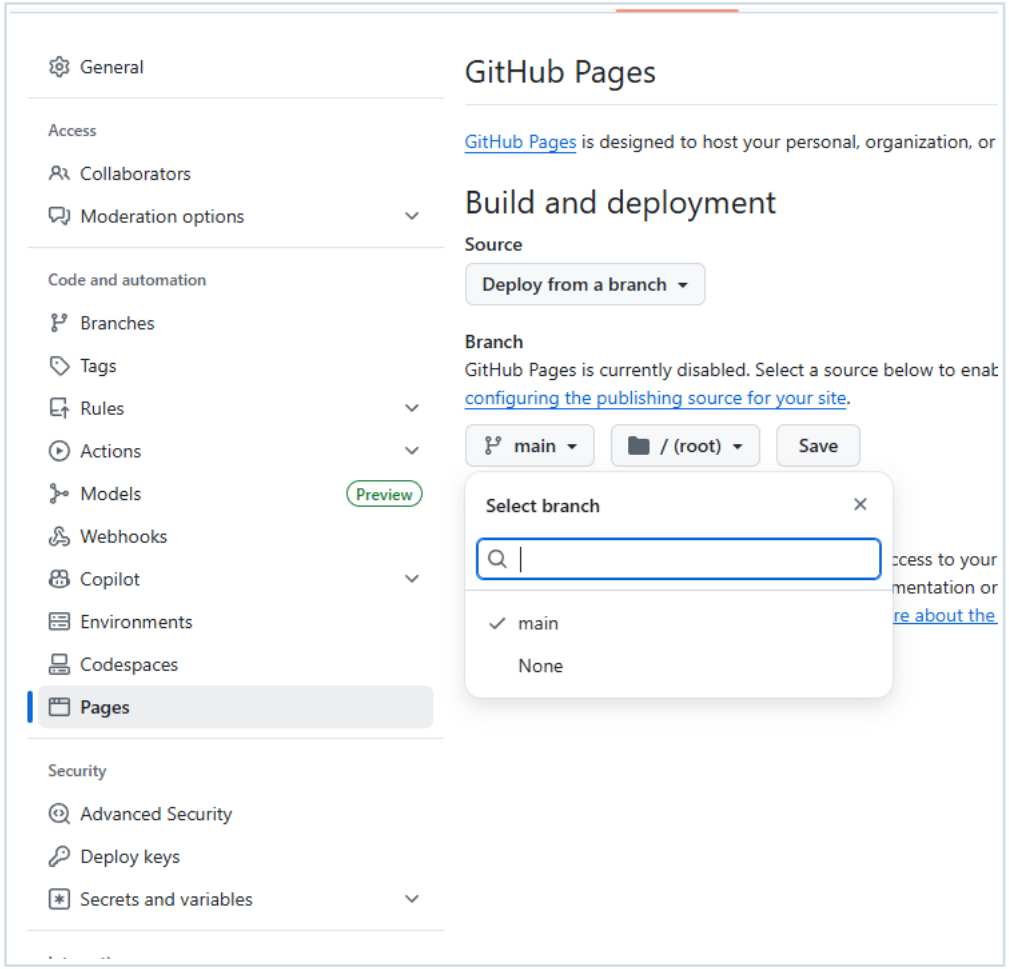
저장소의 Settings에서 한 번 켜 두면, 그다음부터는 main에 push할 때마다 GitHub가 다시 배포합니다.

안내 GitHub Docs — Creating a GitHub Pages site (영문)

1	**Settings**	저장소 위쪽 탭의 Settings (gear 그림)
2	**Pages**	왼쪽 메뉴에서 Pages
3	**Source**	Build and deployment의 Source를 Deploy from a branch로
4	**Branch**	main · /(root)를 고르고 Save

```
https://사용자명.github.io/portfolio/
/* 저장소 이름이 주소 뒤에 붙습니다 — 대소문자까지 같아야 열립니다 */
/* push 뒤 반영까지 몇 분 걸릴 수 있습니다 (문서 : 최대 10분) */
```

Pages 설정 화면입니다



고를 것

Source — Deploy from a branch
Branch — main, 폴더 — / (root)
그다음 Save

주소 확인

잠시 뒤 Pages 화면에 다시 들어가면
「Your site is live at」 줄에
공개 주소가 나옵니다.

1기 수업 화면(2025년) — 메뉴 이름이 조금 다를 수 있음

실습 03

포트폴리오를 GitHub Pages로 공개하기

DAY 06 최종본을 oz-lecture 밖 portfolio 폴더로 옮겨 새 저장소로 만들고, Pages로 공개합니다.
완성한 주소를 채팅창에 공유하고 휴대폰으로도 열어 봅니다.

```
# oz-lecture 옆에 portfolio 폴더를 만들고 Day06의
# about-me.html · style.css · me.jpg를 복사 → index.html로 이름 바꾸기
$ cd portfolio
$ git init
$ git add .
$ git commit -m "포트폴리오 첫 화면 추가"
$ git remote add origin https://github.com/사용자명/portfolio.git
$ git push -u origin main
# GitHub : Settings → Pages → Deploy from a branch
#          → main · /(root) → Save
```

확인할 것

사용자명.github.io/portfolio/ 가 열린다

사진과 스타일이 보인다

휴대폰에서도 열린다

막히면

404면 몇 분 뒤 다시 열고,
index.html 이름을 확인합니다.

실습 03 · 점검

Pages에서 자주 나오는 세 가지

!

주소가 404예요

몇 분 뒤 다시 엽니다.
index.html이 저장소
맨 위에 있는지 봅니다.

!

스타일이 안 보여요

CSS · 사진 경로와
대소문자를 확인합니다.
Pages는 구분합니다.

!

Pages를 못 켜요

무료 계정은 Public
저장소에서만 Pages를
쓸 수 있습니다.

주소가 열리면 다른 사람의 기기에서도 한 번 열어 봅니다

내 컴퓨터에서만 확인하면 경로 문제를 못 잡습니다. 옆 사람과 주소를 바꿔 열어 보게 하세요.

04

Git 사용 전략

브랜치 · 워크플로 · merge · conflict · Pull Request · AI 초안

브랜치는 작업을 따로 떼어두는 갈래입니다

브랜치 없이

한 줄에 모두 쌓입니다

새 기능을 만들다 망가지면
지금 잘 되던 화면까지 같이 망가집니다.
여러 명이 동시에 고치면 섞입니다.

브랜치를 쓰면

따로 만들고 나중에 합칩니다

main은 그대로 두고 옆에서 작업합니다.
다 되면 main에 합칩니다.
잘못되면 그 갈래만 버리면 됩니다.

main은 언제든 정상으로 열리는 상태로 둡니다

실험은 브랜치에서 하고, 확인이 끝난 것만 main에 합칩니다.

04 · 워크플로

브랜치를 쓰는 약속을 워크플로라고 합니다

1기 자료의 정의 — 「Git을 사용해 코드를 체계적으로 관리하는 규칙」

01

작업 분리

사람마다, 기능마다
브랜치를 나눠 일합니다.

02

체계적 병합

정한 순서와 방법으로
main에 합칩니다.

03

추적과 충돌 줄이기

누가 무엇을 바꿨는지 남고,
충돌이 줄어듭니다.

팀이 커질수록 어디서 작업하고 언제 합칠지를 먼저 정합니다

이 약속을 워크플로 또는 브랜치 전략이라고 부릅니다. 대표적인 세 가지를 봅니다.

main 하나에 바로 쌓는 Centralized



모든 커밋이 main 한 줄에 쌓입니다

특징

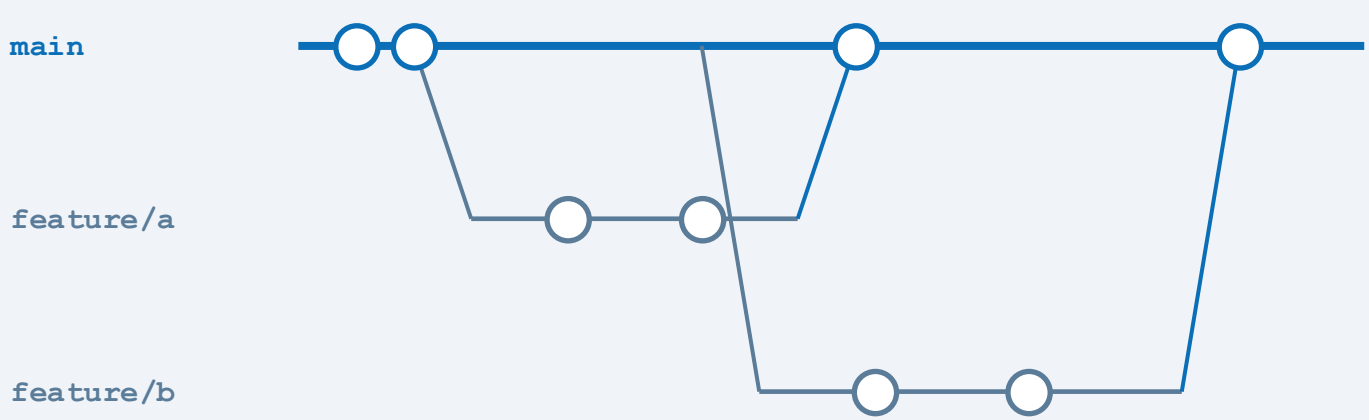
단일 브랜치(main) 중심 —
모두 main에 바로 커밋합니다.

장점과 단점

간단하고 배우기 쉽지만
여럿이 쓰면 충돌이 잦습니다.

혼자 쓰는 저장소나 아주 작은 팀에 맞습니다 — 오늘 두 저장소가 이 방식입니다

기능마다 갈래를 내는 Feature Branch



기능마다 갈라졌다가, 검토를 거쳐 main에 합칩니다

특징

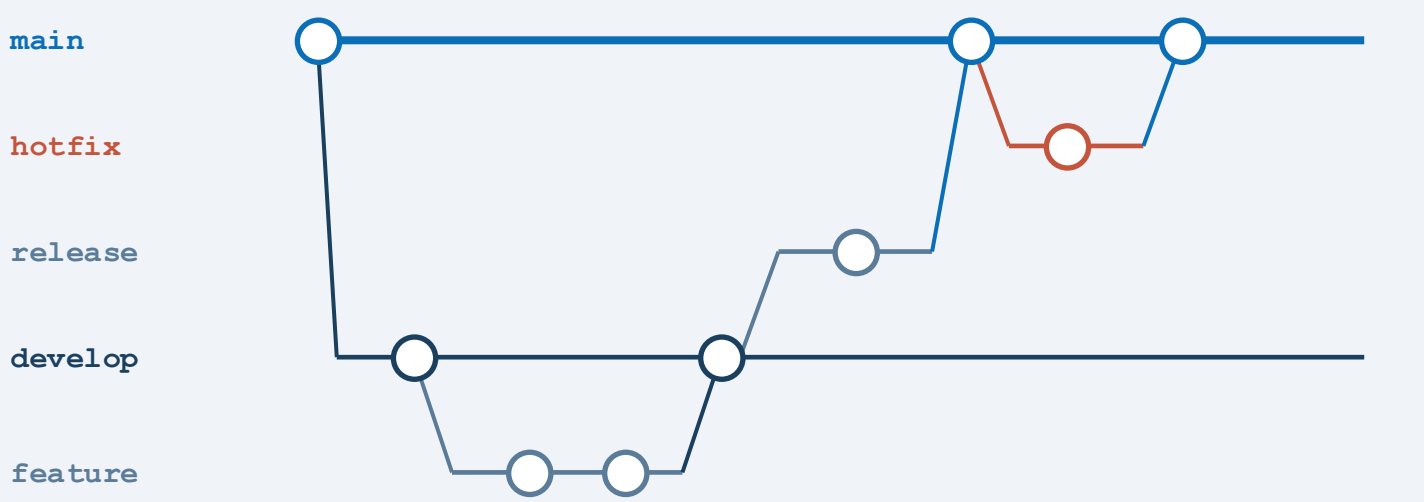
기능마다 feature/* 브랜치,
다 되면 PR로 main에 합칩니다.

장점과 단점

코드 검토가 쉽고 협업에 강하지만
브랜치 관리가 필요합니다.

여러 사람이 한 저장소를 쓰는 팀 프로젝트에 맞습니다

줄기가 다섯인 Git Flow



main · develop을 두고 feature · release · hotfix를 더합니다

특징

main · develop 두 줄기에
feature · release · hotfix

장점과 단점

작업 분리와 통합이 체계적이지만
처음엔 복잡하고 규칙이 필요합니다.

배포 일정이 있는 큰 프로젝트에서 씁니다 — 오늘은 이름과 모양만 알아둡니다

규모에 맞는 전략을 고릅니다

전략	브랜치	맞는 경우	우리 수업
Centralized	main 하나	혼자 · 아주 작은 팀	오늘 두 저장소
Feature Branch	main + feature/*	여럿이 기능을 나눌 때	랜딩페이지 팀 프로젝트
Git Flow	main · develop · feature · release · hotfix	배포 일정이 있는 큰 프로젝트	이름과 모양만

전략은 명령어가 아니라 팀의 약속입니다

시작하기 전에 함께 정하고, 정한 대로 브랜치를 나누고 합칩니다.

브랜치 이름에는 관례가 있습니다

이름	역할	예시
`main`	배포용 최종 버전	언제나 정상으로 열리는 상태
`develop`	개발 중 통합 버전	기능을 모아 확인하는 자리
`feature/*`	새 기능 개발	feature/login, feature/gnb
`release/*`	배포 준비	release/v1.0.1
`hotfix/*`	긴급 수정	hotfix/bug-135

오늘은 main 하나, 팀 프로젝트에서는 main과 feature/*를 씁니다

나머지는 Git Flow를 쓰는 팀에서 만나는 이름입니다. 형태만 알아둡니다.

04 · 운영

브랜치를 쓸 때 지킬 세 가지

01

이름으로 목적을 알린다

feature-contact 처럼
무엇을 하는 갈래인지
이름에서 보이게 합니다.

02

오래 두지 않는다

갈라진 채로 오래 두면
합칠 때 충돌 지점이
여러 곳에서 생깁니다.

03

자주 합친다

작게 나누어 자주
main에 합칩니다.
한 번에 크게 합치는 것보다
훨씬 안전합니다.

브랜치를 작게 나누고 빨리 합치면 충돌이 줄어듭니다

합치는 것은 받는 쪽에서 합니다

```
$ git switch main                  합칠 곳으로 옮기고
$ git merge feature-contact        가져옵니다
Updating a1b2c3d..e4f5a6b
Fast-forward

$ git branch -d feature-contact    다 쓴 갈래는 지웁니다
```

방향을 헷갈리지 않게

main으로 옮긴 뒤
feature를 merge합니다.
반대로 하면 main이
feature 안으로 들어갑니다.

합친 뒤에는 브라우저에서 화면을 한 번 확인합니다

같은 줄을 서로 다르게 고치면 충돌합니다

```
CONFLICT (content): Merge conflict in index.html
<<<<<<< HEAD
  <h1>김코딩의 포트폴리오</h1>          ← main 쪽 내용
=======
  <h1>김코딩 소개합니다</h1>            ← 가져온 쪽 내용
>>>>>>> feature-title
/* 이 표시가 파일 안에 그대로 들어갑니다 */
```

오류가 아닙니다

Git이 판단할 수 없으니
사람에게 물어보는 것입니다.

충돌은 두 쪽이 같은 자리를 서로 다르게 고쳤을 때 생깁니다

멀리 떨어진 줄은 대부분 Git이 알아서 합칩니다. 붙어 있는 줄이나, 한쪽은 고치고 한쪽은 지운 파일은 충돌합니다.

04 · 해결

충돌은 이 순서로 풉니다

1. **01** 파일을 엽니다. 표시가 있는 곳이 충돌 지점입니다
2. **02** 어느 쪽을 남길지 정합니다. 둘을 합쳐도 됩니다
3. **03** `<<<<<<<` `=======` `>>>>>>>` 세 줄을 모두 지웁니다
4. **04** 저장하고 git add 파일명
5. **05** git commit 으로 마무리합니다

표시를 지우지 않고 커밋하면 그 글자가 화면에 그대로 보입니다

PR은「합쳐주세요」라고 요청하는 것입니다

1. **01** 브랜치에서 작업하고 커밋합니다
2. **02** git push -u origin 브랜치이름 으로 올립니다
3. **03** GitHub에 뜨는 Compare & pull request 버튼을 누릅니다
4. **04** 제목과 설명을 적고 Create pull request
5. **05** 동료가 보고 승인하면 Merge pull request

PR은 Feature Branch 전략의 중심입니다

합치기 전에 동료가 보고 의견을 남기는 자리입니다. 혼자 하는 프로젝트에서도 PR을 쓰면 기록이 잘 남습니다.

04 · PR 설명

PR 설명에는 세 가지를 적습니다

이렇게 쓰면

읽는 사람이 다시 물어봅니다

```
수정했습니다
확인 부탁드립니다
```

이렇게 쓰세요

무엇을 · 왜 · 어떻게 확인

```
## 무엇을
연락처 섹션을 추가했습니다
## 왜
메일 주소가 없었습니다
## 확인
좁은 화면 줄바꿈
```

이 세 항목은 DAY 22 랜딩페이지 팀 프로젝트에서 그대로 씁니다

커밋 메시지와 PR 초안은 AI에게 맡길 수 있습니다

```
$ git diff --staged
/* 스테이지에 담긴 변경 내용을 그대로 보여줍니다 */

아래 diff를 보고 커밋 메시지 후보를 세 개 써 주세요.
한 줄, 무엇을 했는지 중심으로 써 주세요.
[여기에 diff를 붙입니다]
```

diff를 함께 줍니다

코드를 안 보여주고 물으면 일반적인 문장만 돌아옵니다. 무엇이 바뀌었는지 보여줘야 그 내용으로 써 줍니다.

후보를 받아 고르고 고치는 것까지가 내 일입니다

PR 설명도 같은 방식으로 초안을 받을 수 있습니다. 다만 확인은 사람이 합니다.

04 · 혼자 더 해 보기

브랜치는 그림으로 익히는 편이 빠릅니다

명령어만으로는 갈래가 어떻게 갈라지고 합쳐지는지 머릿속에 그려지지 않습니다.

연습 Learn Git Branching — 명령어를 치면 그림이 움직이는 연습 사이트

이렇게 진행합니다

함께 읽고 먼저 말해보기

강사 화면에 띄우고 함께 읽기
다음에 칠 명령어를 먼저 말해보게 하기
틀리면 그림에서 무엇이 달라졌는지 보기

여기까지 나오는 것

오늘 본 그림 그대로

branch와 switch
merge
원격 저장소와 push · pull

참고 GitHub Git 치트시트 (한국어)

점검

Git에서 자주 나오는 세 가지

!

push가 거절돼요

원격에 내가 모르는
커밋이 있는 경우입니다.
먼저 git pull 합니다.

!

파일이 안 올라가요

.gitignore에 걸렸거나
add를 안 한 경우입니다.
git status로 확인합니다.

!

실수로 커밋했어요

아직 push 전이라면
git reset --soft HEAD~1
로 되돌릴 수 있습니다.

막히면 git status 를 먼저 칩니다

지금 어느 브랜치에 있고 어느 파일이 어느 자리에 있는지를 알면 다음에 칠 명령어가 정해집니다.

WRAP UP

오늘 정리

개념

Git은 변경 이력을 기록하는 도구다

파일은 작업 폴더 → 스테이지 → 저장소를 지난다

커밋 하나에는 한 가지 일만 담는다

push는 올리기, pull은 내려받기다

저장소 하나에는 프로젝트 하나를 둔다

브랜치 전략은 팀의 약속이다

직접 쓸 수 있어야 하는 것

init으로 저장소 만들기

add와 commit으로 기록 남기기

GitHub에 push하고 README.md 쓰기

GitHub Pages로 포트폴리오 공개하기

규모에 맞는 브랜치 전략 고르기

충돌 표시를 읽고 풀기

NEXT

DAY 08

Bootstrap 5 개념 및 기본 사용법

오늘 올린 페이지의 CSS는 전부 직접 쓴 것입니다.
다음 시간에는 Bootstrap을 얹어 같은 화면을
훨씬 적은 코드로 만들어 봅니다.

수고하셨습니다.